# Organization of Records into Blocks

## Lecture 2: Physical Storage, Blocking Factor, Spanning and Disk I/O

Department of Computer Science and Engineering

Graphic Era (Deemed to be University)

# Recap: File Organization

## From Lecture 1

A file is a collection of related records.

- A field stores one fact.
- A record contains related fields.
- A file contains related records.
- File organization determines how records are logically arranged and accessed.

## Today's Question

How are these logical records grouped into physical storage units on a disk?

# Why Organize Records into Blocks?

- Disk storage is accessed in units called blocks.
- Reading one record may involve reading an entire block.
- Multiple records can be stored together in a block.
- Grouping records can reduce the number of disk I/O operations required for sequential processing.
- Efficient block utilization reduces wasted storage.

## Key Principle

File performance depends not only on record organization, but also on how records are placed inside storage blocks.

# Logical Record vs Physical Record

## Logical Record

- Unit meaningful to the application.
- Contains related fields.
- Example: one student's information.

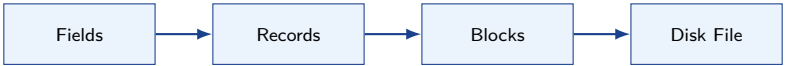
## Physical Record / Block

- Unit of physical storage transfer.
- May contain multiple logical records.
- Example: a 4096-byte disk block.

## Terminology

The term *physical record* is sometimes used for a block in traditional file-processing literature.

# From Fields to Disk Blocks




## Example

- One student record occupies 100 bytes.
- One disk block has 512 bytes available.
- Five complete records can fit in one block.
- Twelve records require three blocks.

# What Is a Disk Block?

## Definition

A disk block is a fixed-size unit of storage used for transferring data between secondary storage and memory.

Common illustrative block sizes include:

- 512 bytes
- 1024 bytes
- 2048 bytes
- 4096 bytes

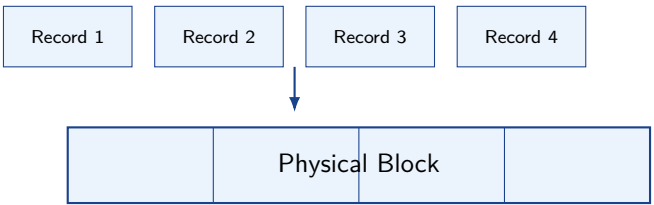
## Important

Actual block sizes depend on the storage system, file system, and database configuration.

# What Is Blocking?

## Definition

Blocking is the process of grouping logical records into physical storage blocks.




- Blocking improves transfer efficiency.
- A block may contain several records.

# Blocking and Unblocking

## Blocking

1. Obtain logical records.
2. Pack records into a buffer.
3. Write the buffer as a block.

## Unblocking

1. Read a physical block.
2. Identify its records.
3. Deliver records to the application.

## Example

A file-processing program reads a 4096-byte block and extracts its individual student records.

# Benefits of Blocking

1. Fewer disk read/write operations.
2. Better sequential processing throughput.
3. More efficient use of I/O buffers.
4. Reduced per-record I/O overhead.
5. Suitable for large file processing.

**Trade-off**

Larger blocks can reduce I/O count but may transfer unneeded data during individual-record access.

# Fixed-Length Records

## Definition

Every record occupies the same number of bytes.




- Simple address calculation.
- Easy direct access.
- May waste space in partially used fields.

# Variable-Length Records

## Definition

Records may occupy different amounts of storage.




- Useful for variable-size fields.
- Can reduce unnecessary padding.
- Requires record boundaries or length information.
- More complex than fixed-length storage.

# Fixed vs Variable-Length Records

| Feature | Fixed-Length | Variable-Length |
|---|---|---|
| Size | Same for all records | May differ |
| Direct access | Simple offset calculation | Requires location information |
| Space efficiency | May include padding | Can reduce padding |
| Implementation | Relatively simple | More complex |
| Example | Fixed-size C structure | Variable-length text fields |

# Blocking Factor (BFR)

## Definition

The blocking factor is the number of complete, fixed-length records that fit in one block.

For unspanned fixed-length records:

$$\boxed{BFR = \left\lfloor \frac{B}{R} \right\rfloor}$$

where:

- $B$ = usable block size in bytes
- $R$ = record size in bytes

## Condition

This formula assumes there is no additional per-record or per-block metadata within $B$.

# Blocking Factor: Example 1

## Given

$$B = 1024 \text{ bytes}, \qquad R = 100 \text{ bytes}$$

$$BFR = \left\lfloor \frac{1024}{100} \right\rfloor$$

$$= \lfloor 10.24 \rfloor$$

$$\boxed{BFR = 10}$$

## Interpretation

Each block stores 10 complete records, leaving 24 bytes unused.

# Blocking Factor: Example 2

## Given

$$B = 4096 \text{ bytes}, \qquad R = 256 \text{ bytes}$$

$$BFR = \left\lfloor \frac{4096}{256} \right\rfloor$$

$$\boxed{BFR = 16}$$

Unused space:

$$4096 - (16 \times 256) = 0$$

## Observation

All usable block space is occupied by records.

# Unused Space per Block

For unspanned fixed-length records:

$$\boxed{W = B - (BFR \times R)}$$

where $W$ is unused space in a filled block.

## Example

$$B = 1000, \quad R = 180$$

$$BFR = \left\lfloor \frac{1000}{180} \right\rfloor = 5$$

$$W = 1000 - (5 \times 180)$$

$$\boxed{W = 100 \text{ bytes}}$$

# Block Utilization

## Definition

Block utilization measures the fraction of usable block capacity occupied by record data.

For a full unspanned block:

$$\boxed{U = \frac{BFR \times R}{B} \times 100\%}$$

## Example

$$B = 1000, \quad R = 180, \quad BFR = 5$$

$$U = \frac{5 \times 180}{1000} \times 100$$

$$\boxed{U = 90\%}$$

# Unspanned Record Organization

## Definition

An unspanned record must fit entirely within one block.

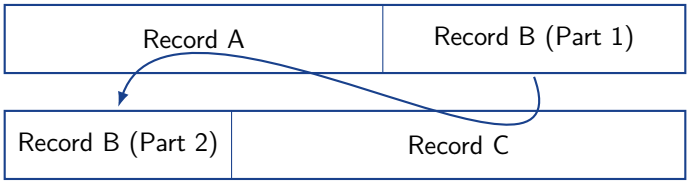



- Record boundaries never cross block boundaries.
- Easy record retrieval.
- May leave unused space.
- Requires each record to fit within one block.

# Spanned Record Organization

## Definition

A spanned record may be divided across two or more blocks.




- Records can cross block boundaries.
- Reduces some forms of unused space.
- Requires information to reconstruct records.

# Spanned vs Unspanned

| Feature | Unspanned | Spanned |
|---|---|---|
| Record boundary | Within one block | May cross blocks |
| Space usage | May waste block space | Can use space more fully |
| Record retrieval | Simpler | May require multiple blocks |
| Large records | Cannot exceed block capacity | Can exceed one block |
| Implementation | Simpler | More complex |

# Worked Example: Unspanned vs Spanned

## Given

$$B = 1000, \quad R = 600, \quad n = 10$$

**Unspanned:**

$$BFR = \left\lfloor \frac{1000}{600} \right\rfloor = 1$$

$$b = \left\lceil \frac{10}{1} \right\rceil = \boxed{10 \text{ blocks}}$$

**Ideal spanned packing, ignoring metadata:**

$$b = \left\lceil \frac{10 \times 600}{1000} \right\rceil = \boxed{6 \text{ blocks}}$$

## Observation

Spanning may reduce the required number of blocks, but a record may need data from multiple blocks.

# Number of Blocks: Unspanned Files

For $n$ fixed-length records:

$$\boxed{b = \left\lceil \frac{n}{BFR} \right\rceil}$$

where:

- $b$ = number of blocks
- $n$ = number of records
- $BFR$ = blocking factor

## Why Ceiling?

Even a partially filled final block occupies one complete allocated block.

# Numerical Problem 1

## Question

A file contains 1000 records.
Each record occupies 100 bytes.
Block size is 1024 bytes.
Find the blocking factor and number of blocks.

$$BFR = \left\lfloor \frac{1024}{100} \right\rfloor = 10$$

$$b = \left\lceil \frac{1000}{10} \right\rceil$$

$$\boxed{b = 100 \text{ blocks}}$$

# Numerical Problem 2

## Question

A file contains 2500 records.
Record size = 150 bytes.
Block size = 4096 bytes.

$$BFR = \left\lfloor \frac{4096}{150} \right\rfloor = \boxed{27}$$

$$b = \left\lceil \frac{2500}{27} \right\rceil$$

$$\boxed{b = 93 \text{ blocks}}$$

Unused space in a full block:

$$4096 - (27 \times 150) = \boxed{46 \text{ bytes}}$$

# Numerical Problem 3

## Question

A file has 600 records, each 200 bytes long. The block size is 1024 bytes.
Calculate:

1. Blocking factor
2. Number of blocks
3. Unused space per full block

$$BFR = \lfloor 1024/200 \rfloor = \boxed{5}$$

$$b = \lceil 600/5 \rceil = \boxed{120}$$

$$W = 1024 - (5 \times 200) = \boxed{24 \text{ bytes}}$$

# Numerical Problem 4: Block Header

## Question

A 4096-byte block reserves 96 bytes for a header and other block metadata.
Each record occupies 250 bytes.
Find the blocking factor.

Usable space:

$$B = 4096 - 96 = 4000$$

$$BFR = \left\lfloor \frac{4000}{250} \right\rfloor$$

$$\boxed{BFR = 16}$$

## Lesson

Subtract reserved block metadata before calculating the blocking factor.

# Why Disk I/O Matters

- Disk access is generally much more expensive than simple CPU operations.
- Reading multiple records together can reduce the number of transfers.
- Sequential scans benefit from block buffering.
- Poor block utilization increases the number of blocks to be transferred.

## Simplified I/O Cost Model

For a full sequential file scan:

$$\boxed{\text{Block reads} \approx b}$$

where $b$ is the number of occupied data blocks.

# Example: I/O Cost

Suppose a file contains 10,000 records.

## One Record per Block

$$BFR = 1$$

$$b = 10000$$

$$\boxed{\text{10000 reads}}$$

## Ten Records per Block

$$BFR = 10$$

$$b = 1000$$

$$\boxed{\text{1000 reads}}$$

## Result

The block-read count decreases by a factor of 10 under the simplified sequential-scan model.

# Blocking Factor and Search

## Sequential Search

A sequential search may require reading many blocks before locating the desired record.

For $b$ data blocks:

- Best case: first block.
- Worst case: up to $b$ block reads.
- Average case: approximately $(b+1)/2$ block reads for a uniformly located successful search in equally occupied blocks.

## Note

This is an idealized model that ignores caching, prefetching, and other storage-system effects.

# Block-Based Storage in C

We can model a block as a structure containing a fixed number of records.

## Design

- Define a record structure.
- Define a block structure.
- Store multiple records in each block.
- Write or read blocks using binary file I/O.

## Illustrative Model

Each block contains at most four student records.

# C Program: Record and Block Structures

```c
#include <stdio.h>

#define BFR 4

typedef struct {
    int roll;
    char name[20];
    float marks;
} Student;

typedef struct {
    int count;
    Student records[BFR];
} Block;
```

## Explanation

- `Student` represents a logical record.
- `Block` contains up to four records.
- `count` stores the number of valid records.

## Note

This is a teaching model. Its physical block size is `sizeof(Block)`, including any padding.

# C Program: Writing Blocks

```c
#include <stdio.h>

#define BFR 4

typedef struct {
    int roll;
    char name[20];
    float marks;
} Student;

typedef struct {
    int count;
    Student records[BFR];
} Block;

int main(void) {
    FILE *fp = fopen("blocks.dat", "wb");
    Block b = {0};

    if (fp == NULL) {
        perror("File error");
        return 1;
    }

    b.count = 2;
    b.records[0] = (Student){101, "Aman", 88.5f};
    b.records[1] = (Student){102, "Riya", 91.0f};

    if (fwrite(&b, sizeof b, 1, fp) != 1) {
        perror("Write failed");
        fclose(fp);
        return 1;
    }

    fclose(fp);
    return 0;
}
```

# C Program: Reading Blocks

```c
#include <stdio.h>

#define BFR 4

typedef struct {
    int roll;
    char name[20];
    float marks;
} Student;

typedef struct {
    int count;
    Student records[BFR];
} Block;

int main(void) {
    FILE *fp = fopen("blocks.dat", "rb");
    Block b;

    if (fp == NULL) {
        perror("File error");
        return 1;
    }

    while (fread(&b, sizeof b, 1, fp) == 1) {
        if (b.count < 0 || b.count > BFR) {
            puts("Invalid block");
            break;
        }

        for (int i = 0; i < b.count; i++)
            printf("%d %s %.2f\n",
                   b.records[i].roll,
                   b.records[i].name,
                   b.records[i].marks);
    }
```

# How the C Example Works

1. The program defines a fixed-capacity block.
2. Each block can contain four student records.
3. The `count` field identifies valid records.
4. The block is written using `fwrite()`.
5. The block is read using `fread()`.
6. Individual records are extracted from the block.

**Limitation**

Writing raw structures is appropriate for a simple same-platform demonstration, but portable file formats require explicitly defined serialization.

# Classroom Exercise 1

## Question

A file contains 1800 fixed-length records.
Each record occupies 125 bytes.
The usable block size is 2048 bytes.
Find:

1. Blocking factor
2. Number of blocks
3. Unused space per full block

**Try solving before viewing the answer.**

# Classroom Exercise 1: Solution

$$BFR = \left\lfloor \frac{2048}{125} \right\rfloor = \boxed{16}$$

$$b = \left\lceil \frac{1800}{16} \right\rceil = \boxed{113}$$

$$W = 2048 - (16 \times 125)$$

$$\boxed{W = 48 \text{ bytes}}$$

## Final Block

112 full blocks store 1792 records.
The final block stores 8 records.

# Classroom Exercise 2

## Question

A file contains 500 records of 300 bytes each.
Block size = 1024 bytes.
Compare the number of blocks required under unspanned and ideal spanned storage.

**Unspanned:**

$$BFR = \lfloor 1024/300 \rfloor = 3$$

$$b = \lceil 500/3 \rceil = \boxed{167}$$

**Ideal spanned:**

$$b = \left\lceil \frac{500 \times 300}{1024} \right\rceil = \boxed{147}$$

# MCQ 1

**What does blocking factor represent?**

1. Number of files in a directory
2. Number of records that fit in a block
3. Number of fields in a record
4. Number of disks in a system

## Correct Answer

Option 2

# MCQ 2

**Which statement is correct for unspanned record organization?**

1. A record may cross several blocks.
2. Records have no boundaries.
3. A record must fit within one block.
4. Every block stores exactly one record.

## Correct Answer

Option 3

# MCQ 3

**If $B = 1000$ bytes and $R = 250$ bytes, what is the blocking factor?**

1. 2
2. 3
3. 4
4. 5

## Correct Answer

Option 3: $BFR = 4$

# MCQ 4

**Which organization allows a record to continue into another block?**

1. Unspanned
2. Spanned
3. Sequential-only
4. Relative-only

## Correct Answer

Option 2: Spanned

# MCQ 5

**For $n$ fixed-length records and blocking factor $BFR$, how many unspanned blocks are needed?**

1. $n \times BFR$
2. $\lfloor n/BFR \rfloor$
3. $\lceil n/BFR \rceil$
4. $BFR/n$

## Correct Answer

Option 3

# Important Formulas

## Blocking Factor

$$BFR = \left\lfloor \frac{B}{R} \right\rfloor$$

## Number of Unspanned Blocks

$$b = \left\lceil \frac{n}{BFR} \right\rceil$$

## Unused Space per Full Block

$$W = B - (BFR \times R)$$

## Block Utilization

$$U = \frac{BFR \times R}{B} \times 100\%$$

# Lecture Summary

- Logical records are grouped into physical blocks.
- Blocking reduces the number of I/O transfers.
- Fixed-length records simplify storage calculations.
- Variable-length records need boundary information.
- Blocking factor measures records per block.
- Unspanned records remain within one block.
- Spanned records may cross block boundaries.
- Block utilization affects storage efficiency.
- The number of blocks influences file-scan I/O cost.

# Questions for Revision

1. Define a logical record and physical block.
2. What is blocking and why is it useful?
3. Differentiate fixed- and variable-length records.
4. Derive the blocking factor formula.
5. Compare spanned and unspanned organization.
6. Calculate the number of blocks for a given file.
7. Explain block utilization.
8. Describe how block organization affects disk I/O.
9. Write a C program to store multiple records inside a block.

# References


1. Reema Thareja, *Data Structures Using C*, Chapter 16: Files and Their Organization.
2. Ramez Elmasri and Shamkant B. Navathe, *Fundamentals of Database Systems*, topics on disk storage and record organization.
3. Abraham Silberschatz, Henry F. Korth, and S. Sudarshan, *Database System Concepts*, topics on storage and file organization.

# Student Feedback

## Share Your Feedback



Thank you!